Practical 7

Aim

To implement a Doubly Linked List in C++ and perform the following operations:

- Creation
- Insertion
- Deletion
- Traversal (Display)
- Search
- Reverse

Theory

A Doubly Linked List (DLL) is a dynamic linear data structure in which each node contains three parts:

- Previous Pointer (prev) – Stores the address of the previous node.
- Data – Stores the value.
- Next Pointer (next) – Stores the address of the next node.

Unlike a Singly Linked List, a Doubly Linked List allows traversal in both forward and backward directions because each node maintains links to both its previous and next nodes.

Memory for each node is allocated dynamically during program execution, making insertion and deletion operations more efficient as compared to arrays.

The first node is called the Head, whose prev pointer is NULL, while the last node has its next pointer set to NULL.

Representation
NULL ← [10] ⇄ [20] ⇄ [30] → NULL
↑↑
Head Tail

Basic Operations

- Creation: Create nodes dynamically and link them together.
- Insertion: Insert a new node at the beginning or end of the list.
- Deletion: Delete a node by its value.

- Traversal: Display all nodes from head to tail.
- Search: Find a particular element in the list.
- Reverse: Reverse the linked list by changing the links.

Applications

- Browser Forward and Backward Navigation

- Undo and Redo Operations
- Music and Video Playlists
- Image Viewers
- Memory Management
- Implementation of Deque

Code implementation

```
#include <iostream> using namespace std;

class Node
{
public:
int data; Node *prev; Node *next;

Node(int value)
{
data = value; prev = NULL; next = NULL;
}
};

class DoublyLinkedList
{
private:
Node *head;

public:
// Creation DoublyLinkedList()
{
head = NULL;
}

// Insert at Beginning
void insertBeginning(int value)
{

Node *newNode = new Node(value);

if (head == NULL)
{
head = newNode; return;
}

newNode->next = head; head->prev = newNode; head = newNode;
}

// Insert at End
void insertEnd(int value)
{
```

```
Node *newNode = new Node(value);

if (head == NULL)
{
head = newNode; return;
}

Node *temp = head;

while (temp->next != NULL)
{
temp = temp->next;
}

temp->next = newNode; newNode->prev = temp;
}

// Delete Node
void deleteNode(int key)
{
if (head == NULL)
{
cout << "List is Empty\n"; return;
}

Node *temp = head;

while (temp != NULL && temp->data != key)
{
temp = temp->next;

}

if (temp == NULL)
{
cout << "Element not found.\n"; return;
}

if (temp == head)
{
head = head->next;

if (head != NULL) head->prev = NULL;

delete temp;
cout << key << " deleted successfully.\n"; return;
}

if (temp->next != NULL)
```

```
temp->next->prev = temp->prev;

if (temp->prev != NULL)
temp->prev->next = temp->next; delete temp;
cout << key << " deleted successfully.\n";
}

// Traverse void display()
{
if (head == NULL)
{
cout << "List is Empty\n"; return;
}

Node *temp = head;
cout << "Doubly Linked List: "; while (temp != NULL)
{
cout << temp->data << " <-> "; temp = temp->next;
}


cout << "NULL\n";
}

// Search
void search(int key)
{
Node *temp = head;

while (temp != NULL)
{
if (temp->data == key)
{
cout << key << " found in the list.\n"; return;
}

temp = temp->next;
}

cout << key << " not found.\n";
}

// Reverse void reverse()
{
if (head == NULL)
{
cout << "List is Empty\n"; return;
}
```

```
Node *current = head; Node *temp = NULL;

while (current != NULL)
{
temp = current->prev;
current->prev = current->next; current->next = temp;
current = current->prev;
}

if (temp != NULL) head = temp->prev;

cout << "Doubly Linked List Reversed Successfully.\n";
}
};


int main()
{
DoublyLinkedList list; int choice, value;
do
{
cout << "\n----- DOUBLY LINKED LIST MENU      \n";
cout << "1. Insert at Beginning\n"; cout << "2. Insert at End\n";
cout << "3. Delete\n"; cout << "4. Display\n"; cout << "5. Search\n"; cout << "6. Reverse\n"; cout << "7.
Exit\n";

cout << "Enter Choice: "; cin >> choice;

switch (choice)
{
case 1:
cout << "Enter Value: "; cin >> value; list.insertBeginning(value); break;

case 2:
cout << "Enter Value: "; cin >> value; list.insertEnd(value); break;

case 3:
cout << "Enter Value to Delete: "; cin >> value; list.deleteNode(value);
break;

case 4:
list.display(); break;

case 5:
cout << "Enter Value to Search: "; cin >> value;
list.search(value);

break;
```

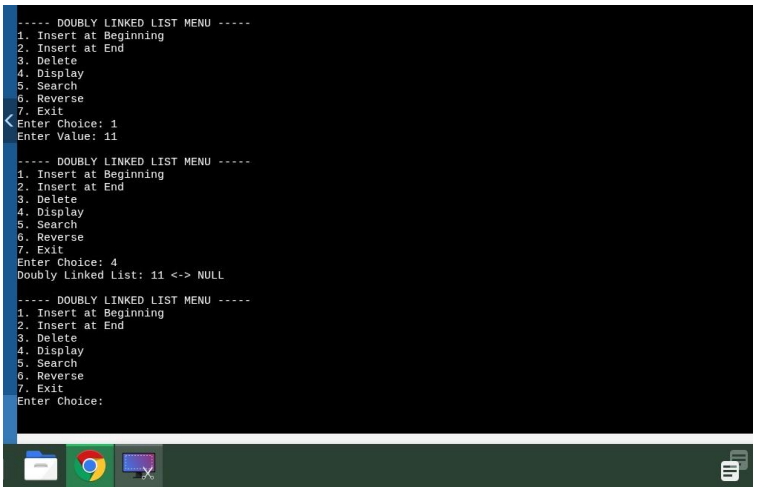
```
case 6:
list.reverse(); break;

case 7:
cout << "Program Ended.\n"; break;

default:
cout << "Invalid Choice\n";
}

} while (choice != 7);

return 0;
}
```



Practical 8

Aim: To implement a Circular Linked List data structure and perform fundamental operations including

1. Creation
2. Insertion
3. Deletion
4. Traversal
5. Search
6. Reversing the list.

Theory:

A **Circular Linked List (CLL)** is a variation of a linked list in which the last node points back to the first node instead of pointing to NULL. This forms a continuous circular loop.

- ### Structure of a Node
  Each node consists of two parts:
  1. data: Stores the actual element/value.
  2. next: Pointer/reference that points to the next node in the list.
- ### Key Operations
  1. **Creation:** Initializing a list node with a value where the next pointer references itself.
  2. **Insertion:** Elements can be inserted at the beginning, at the end, or at a specific position by appropriately re-linking the pointers.
  3. **Deletion:** Removing a node by updating the next pointer of its preceding node to skip the target node.
  4. **Traversal:** Visiting each node starting from the head and stopping once the traversal loops back to the head node.
  5. **Search:** Sequentially scanning nodes from the head until the desired element is found or the list loops back.
  6. **Reverse:** Reversing the direction of next pointers of all nodes so that the list flows in the opposite direction while maintaining its circular structure.

Code Implementation

```
#include <iostream>
using namespace std;

// Structure for a Node
struct Node {
    int data;
    Node* next;

    Node(int val) {
        data = val;
        next = nullptr;
    }

        --
```

```
};

// Class for Circular Linked List
class CircularLinkedList {
private:
    Node* head;

public:
    CircularLinkedList() {
        head = nullptr;
    }

    // 1. Insertion / Creation at End
    void insert(int val) {
        Node* newNode = new Node(val);
        if (head == nullptr) {
            head = newNode;
            newNode->next = head;
            return;
        }
        Node* temp = head;
        while (temp->next != head) {
            temp = temp->next;
        }
        temp->next = newNode;
        newNode->next = head;
    }

    // 2. Traversal / Display
    void display() {
        if (head == nullptr) {
            cout << "List is empty." << endl;
            return;
        }
        Node* temp = head;
        do {
            cout << temp->data << " -> ";
            temp = temp->next;
        } while (temp != head);
        cout << "(head)" << endl;
    }

    // 3. Search Operation
    bool search(int key) {
        if (head == nullptr) {
            cout << "List is empty." << endl;
            return false;
        }
        Node* temp = head;

        --
```

```
    int pos = 1;
    do {
        if (temp->data == key) {
            cout << "Element " << key << " found at position " << pos << "." << endl;
            return true;
        }
        temp = temp->next;
        pos++;
    } while (temp != head);

    cout << "Element " << key << " not found in the list." << endl;
    return false;
}

// 4. Deletion Operation
void deleteNode(int key) {
    if (head == nullptr) {
        cout << "List is empty." << endl;
        return;
    }

    Node* curr = head;
    Node* prev = nullptr;

    // If head node itself holds the key
    if (curr->data == key) {
        // Case 1: Single node in the list
        if (curr->next == head) {
            delete head;
            head = nullptr;
            return;
        }

        // Case 2: Multiple nodes, update last node's pointer
        Node* last = head;
        while (last->next != head) {
            last = last->next;
        }
        last->next = head->next;
        head = head->next;
        delete curr;
        return;
    }

    // Search for the node to delete
    prev = head;
    curr = head->next;
    while (curr != head) {
        if (curr->data == key) {

    --
```

```
            prev->next = curr->next;
            delete curr;
            return;
        }
        prev = curr;
        curr = curr->next;
    }

    cout << "Element " << key << " not found in the list." << endl;
}

// 5. Reverse Operation
void reverse() {
    if (head == nullptr || head->next == head) {
        return; // Empty list or single element
    }

    Node* prev = nullptr;
    Node* current = head;
    Node* nextNode = nullptr;

    do {
        nextNode = current->next;
        current->next = prev;
        prev = current;
        current = nextNode;
    } while (current != head);

    // Adjust original head's next link to point to new head
    head->next = prev;
    head = prev;
}
};

int main() {
    CircularLinkedList cll;

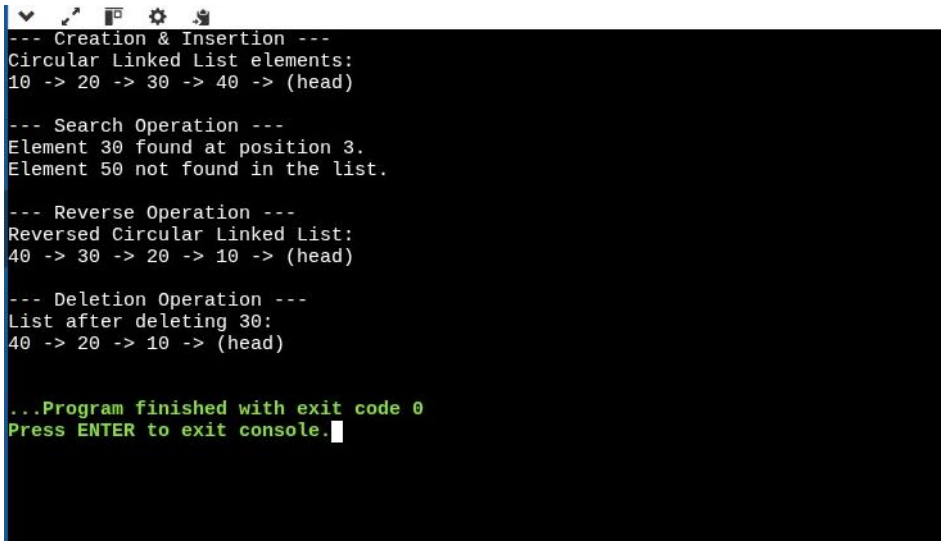
    cout << "--- Creation & Insertion ---" << endl;
    cll.insert(10);
    cll.insert(20);
    cll.insert(30);
    cll.insert(40);
    cout << "Circular Linked List elements:" << endl;
    cll.display();

    cout << "\n--- Search Operation ---" << endl;
    cll.search(30);
    cll.search(50);
```

```
    cout << "\n--- Reverse Operation ---" << endl;
    cll.reverse();
    cout << "Reversed Circular Linked List:" << endl;
    cll.display();

    cout << "\n--- Deletion Operation ---" << endl;
    cll.deleteNode(30);
    cout << "List after deleting 30:" << endl;
    cll.display();

    return 0;
}
```



Practical 9

Aim: To implement
1. Linear Search
2. Binary Search
3. Interpolation Search

algorithms in C++ and analyze their performance across different scenarios.

Theory

Searching is the process of finding the position of a target element within a given dataset. Depending on whether the dataset is sorted or unsorted, different search strategies are applied.

- **1. Linear Search:**
  - **Concept:** Sequentially checks each element in the list from the beginning until a match is found or the end of the list is reached.
  - **Prerequisite:** Works on both sorted and unsorted arrays.
  - **Time Complexity:** Best case O(1), Worst/Average case O(n).
- **2. Binary Search:**
  - **Concept:** Uses a divide-and-conquer approach. It repeatedly divides the search space in half by comparing the target element with the middle element (mid = low + (high - low) / 2).
  - **Prerequisite:** Requires the array to be **sorted**.
  - **Time Complexity:** Best case O(1), Worst/Average case O(log n).
- **3. Interpolation Search:**
  - **Concept:** An improved variant of Binary Search for uniformly distributed data. Instead of always checking the middle, it estimates the position of the target element based on its value using the probe position formula:

    $$Pos=low+\frac{(val-arr[low])*(high-low)}{arr[high]-arr[low]}$$
  - **Prerequisite:** Requires the array to be **sorted** and preferably **uniformly distributed**.
  - **Time Complexity:** Best case O(1), Average case O(log(log n)), Worst case O(n).

Code Implementation

```
#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;

// 1. Linear Search Function
int linearSearch(const vector<int>& arr, int target) {
    for (int i = 0; i < arr.size(); i++) {
        if (arr[i] == target) {


        --
```

```
            return i; // Target found at index i
        }
    }
    return -1; // Target not found
}

// 2. Binary Search Function
int binarySearch(const vector<int>& arr, int target) {
    int low = 0;
    int high = arr.size() - 1;

    while (low <= high) {
        int mid = low + (high - low) / 2;

        if (arr[mid] == target) {
            return mid;
        }
        if (arr[mid] < target) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }
    return -1; // Target not found
}

// 3. Interpolation Search Function
int interpolationSearch(const vector<int>& arr, int target) {
    int low = 0;
    int high = arr.size() - 1;

    while (low <= high && target >= arr[low] && target <= arr[high]) {
        if (low == high) {
            if (arr[low] == target) return low;
            return -1;
        }

        // Estimating the position using interpolation formula
        int pos = low + (((double)(high - low) / (arr[high] - arr[low])) * (target - arr[low]));

        if (arr[pos] == target) {
            return pos;
        }
        if (arr[pos] < target) {
            low = pos + 1;
        } else {
            high = pos - 1;
        }
    }

    --
```

```
    return -1; // Target not found
}

int main() {
    // Unsorted array for Linear Search
    vector<int> unsortedArr = {64, 34, 25, 12, 22, 11, 90};

    // Sorted array for Binary and Interpolation Search
    vector<int> sortedArr = {10, 20, 30, 40, 50, 60, 70, 80, 90, 100};

    int target1 = 22;
    int target2 = 70;
    int target3 = 45; // Element not in list

    cout << "--- 1. Linear Search ---" << endl;
    cout << "Array: ";
    for (int x : unsortedArr) cout << x << " ";
    cout << endl;

    int result = linearSearch(unsortedArr, target1);
    if (result != -1)
        cout << "Element " << target1 << " found at index: " << result << endl;
    else
        cout << "Element " << target1 << " not found." << endl;

    cout << "\n--- 2. Binary Search ---" << endl;
    cout << "Sorted Array: ";
    for (int x : sortedArr) cout << x << " ";
    cout << endl;

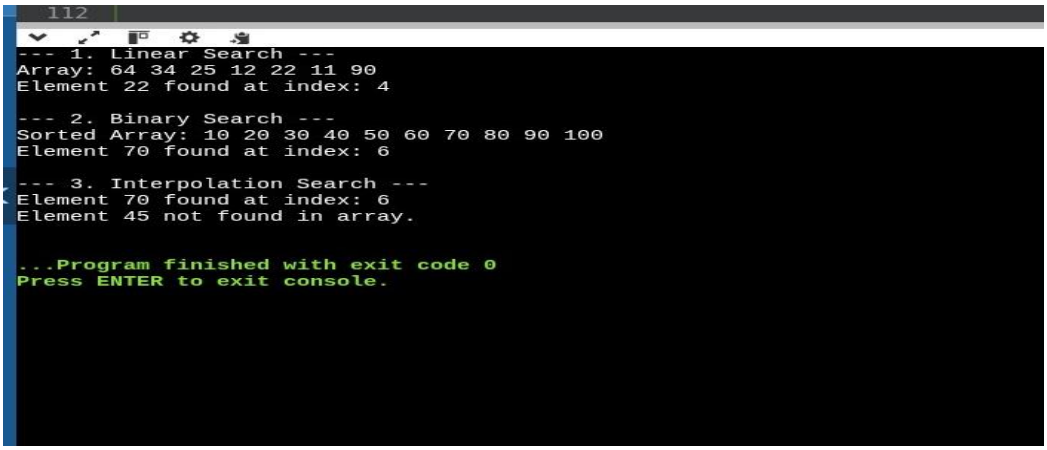
    result = binarySearch(sortedArr, target2);
    if (result != -1)
        cout << "Element " << target2 << " found at index: " << result << endl;
    else
        cout << "Element " << target2 << " not found." << endl;

    cout << "\n--- 3. Interpolation Search ---" << endl;
    result = interpolationSearch(sortedArr, target2);
    if (result != -1)
        cout << "Element " << target2 << " found at index: " << result << endl;
    else
        cout << "Element " << target2 << " not found." << endl;

    // Search for non-existing element
    result = interpolationSearch(sortedArr, target3);
    if (result != -1)
        cout << "Element " << target3 << " found at index: " << result << endl;
    else
        cout << "Element " << target3 << " not found in array." << endl;

    --
```

```
    return 0;
}
```


112
--- 1. Linear Search ---
Array: 64 34 25 12 22 11 90
Element 22 found at index: 4
--- 2. Binary Search ---
Sorted Array: 10 20 30 40 50 60 70 80 90 100
Element 70 found at index: 6
--- 3. Interpolation Search ---
Element 70 found at index: 6
Element 45 not found in array.
...Program finished with exit code 0
Press ENTER to exit console.

Practical 10

Aim: To implement and analyze various sorting algorithms

1. Bubble Sort
2. Selection Sort
3. Insertion Sort
4. Quick Sort
5. Merge Sort

using C++.

Theory

Sorting algorithms are fundamental techniques used to rearrange a given set of data in a specific order (ascending or descending).

1. **Bubble Sort:**
   - **Concept:** Repeatedly compares adjacent elements and swaps them if they are in the wrong order. With each pass, the largest unsorted element "bubbles up" to its correct position at the end.
   - **Time Complexity:** $O(n^2)$ Average/Worst case, O(n) Best case (when already sorted).
   - **Space Complexity:** O(1).
2. **Selection Sort:**
   - **Concept:** Divides the array into a sorted and an unsorted region. It repeatedly finds the minimum element from the unsorted region and places it at the beginning of the unsorted segment.
   - **Time Complexity:** $O(n^2)$ in all cases.
   - **Space Complexity:** O(1).
3. **Insertion Sort:**
   - **Concept:** Builds the sorted array one item at a time by repeatedly taking the next element from the unsorted part and inserting it into its correct position within the sorted part.
   - **Time Complexity:** $O(n^2)$ Average/Worst case, O(n) Best case.
   - **Space Complexity:** O(1).
4. **Quick Sort:**
   - **Concept:** A **Divide and Conquer** algorithm. Selects a 'pivot' element, partitions the array such that elements smaller than the pivot go to its left and larger ones go to its right, and recursively sorts the sub-arrays.
   - **Time Complexity:** O(n log n) Average case, $O(n^2)$ Worst case (e.g., already sorted array with bad pivot).
   - **Space Complexity:** O(log n) (due to recursion stack).
5. **Merge Sort:**
   - **Concept:** A stable **Divide and Conquer** algorithm. Recursively divides the array into two halves until single-element sub-arrays are formed, then merges these sorted sub-arrays back together.
   - **Time Complexity:** O(n log n) across all cases.
   - **Space Complexity:** O(n) (requires auxiliary array for merging).

Code Implementation

```
#include <iostream>
#include <vector>
using namespace std;

// Helper function to print an array
void printArray(const vector<int>& arr) {
    for (int val : arr) {
        cout << val << " ";
    }
    cout << endl;
}

// 1. Bubble Sort
void bubbleSort(vector<int> arr) {
    int n = arr.size();
    for (int i = 0; i < n - 1; i++) {
        bool swapped = false;
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                swap(arr[j], arr[j + 1]);
                swapped = true;
            }
        }
        if (!swapped) break; // Optimization
    }
    cout << "Bubble Sort Result:    ";
    printArray(arr);
}

// 2. Selection Sort
void selectionSort(vector<int> arr) {
    int n = arr.size();
    for (int i = 0; i < n - 1; i++) {
        int minIdx = i;
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[minIdx]) {
                minIdx = j;
            }
        }
        swap(arr[i], arr[minIdx]);
    }
    cout << "Selection Sort Result: ";
    printArray(arr);
}

// 3. Insertion Sort
```

```
void insertionSort(vector<int> arr) {
    int n = arr.size();
    for (int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
    cout << "Insertion Sort Result: ";
    printArray(arr);
}

// 4. Quick Sort Helper Functions
int partition(vector<int>& arr, int low, int high) {
    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;
            swap(arr[i], arr[j]);
        }
    }
    swap(arr[i + 1], arr[high]);
    return i + 1;
}

void quickSort(vector<int>& arr, int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

// 5. Merge Sort Helper Functions
void merge(vector<int>& arr, int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - mid;

    vector<int> L(n1), R(n2);
    for (int i = 0; i < n1; i++) L[i] = arr[left + i];
    for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];

    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {

        --
```

```
            arr[k] = L[i];
            i++;
        } else {
            arr[k] = R[j];
            j++;
        }
        k++;
    }

    while (i < n1) {
        arr[k] = L[i];
        i++;
        k++;
    }
    while (j < n2) {
        arr[k] = R[j];
        j++;
        k++;
    }
}

void mergeSort(vector<int>& arr, int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;
        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}

int main() {
    vector<int> originalArr = {64, 34, 25, 12, 22, 11, 90};

    cout << "Original Array: ";
    printArray(originalArr);
    cout << "----------------------------------------" << endl;

    // 1. Bubble Sort
    bubbleSort(originalArr);

    // 2. Selection Sort
    selectionSort(originalArr);

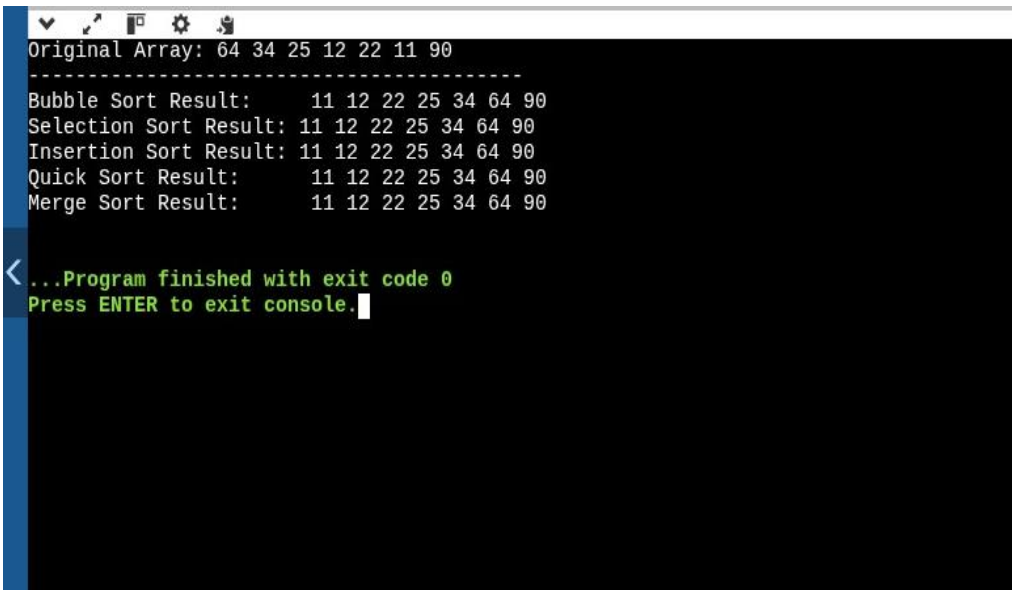
    // 3. Insertion Sort
    insertionSort(originalArr);

    // 4. Quick Sort
    vector<int> qArr = originalArr;
    quickSort(qArr, 0, qArr.size() - 1);
```

```
    cout << "Quick Sort Result:     ";
    printArray(qArr);

    // 5. Merge Sort
    vector<int> mArr = originalArr;
    mergeSort(mArr, 0, mArr.size() - 1);
    cout << "Merge Sort Result:     ";
    printArray(mArr);

    return 0;
}
```


Original Array: 64 34 25 12 22 11 90
Bubble Sort Result: 11 12 22 25 34 64 90
Selection Sort Result: 11 12 22 25 34 64 90
Insertion Sort Result: 11 12 22 25 34 64 90
Quick Sort Result: 11 12 22 25 34 64 90
Merge Sort Result: 11 12 22 25 34 64 90
...Program finished with exit code 0
Press ENTER to exit console.

Practical 11

Aim: To implement a Binary Search Tree (BST) and perform fundamental operations including Creation, Insertion, and Deletion in C++.

Theory:

Sorting algorithms are fundamental techniques used to rearrange a given set of data in a specific order (ascending or descending).

1. **Bubble Sort:**
   - **Concept:** Repeatedly compares adjacent elements and swaps them if they are in the wrong order. With each pass, the largest unsorted element "bubbles up" to its correct position at the end.
   - **Time Complexity:** $O(n^2)$ Average/Worst case, O(n) Best case (when already sorted).
   - **Space Complexity:** O(1).
2. **Selection Sort:**
   - **Concept:** Divides the array into a sorted and an unsorted region. It repeatedly finds the minimum element from the unsorted region and places it at the beginning of the unsorted segment.
   - **Time Complexity:** $O(n^2)$ in all cases.
   - **Space Complexity:** O(1).
3. **Insertion Sort:**
   - **Concept:** Builds the sorted array one item at a time by repeatedly taking the next element from the unsorted part and inserting it into its correct position within the sorted part.
   - **Time Complexity:** $O(n^2)$ Average/Worst case, O(n) Best case.
   - **Space Complexity:** O(1).
4. **Quick Sort:**
   - **Concept:** A **Divide and Conquer** algorithm. Selects a 'pivot' element, partitions the array such that elements smaller than the pivot go to its left and larger ones go to its right, and recursively sorts the sub-arrays.
   - **Time Complexity:** O(n log n) Average case, $O(n^2)$ Worst case (e.g., already sorted array with bad pivot).
   - **Space Complexity:** O(log n) (due to recursion stack).
5. **Merge Sort:**
   - **Concept:** A stable **Divide and Conquer** algorithm. Recursively divides the array into two halves until single-element sub-arrays are formed, then merges these sorted sub-arrays back together.
   - **Time Complexity:** O(n log n) across all cases.
   - **Space Complexity:** O(n) (requires auxiliary array for merging).

Code Implementation

```
#include <iostream>
using namespace std;

// Structure for a Node in the BST
struct Node {
    int data;
    Node* left;
    Node* right;

    Node(int val) {
        data = val;
        left = nullptr;
        right = nullptr;
    }
};

class BinarySearchTree {
public:
    Node* root;

    BinarySearchTree() {
        root = nullptr;
    }

    // 1. Creation / Insertion Operation
    Node* insert(Node* node, int val) {
        // Base case: If tree is empty, return a new node
        if (node == nullptr) {
            return new Node(val);
        }

        // Otherwise, recur down the tree
        if (val < node->data) {
            node->left = insert(node->left, val);
        } else if (val > node->data) {
            node->right = insert(node->right, val);
        }

        // Return the unchanged node pointer
        return node;
    }

    // Helper function to find the minimum value node in a given tree
    Node* findMin(Node* node) {
        Node* current = node;
        // Loop down to find the leftmost leaf


        --
```

```
    while (current && current->left != nullptr) {
        current = current->left;
    }
    return current;
}

// 2. Deletion Operation
Node* deleteNode(Node* root, int key) {
    // Base case
    if (root == nullptr) return root;

    // If key to be deleted is smaller than root, go left
    if (key < root->data) {
        root->left = deleteNode(root->left, key);
    }
    // If key to be deleted is greater than root, go right
    else if (key > root->data) {
        root->right = deleteNode(root->right, key);
    }
    // If key is same as root's data, then this is the node to be deleted
    else {
        // Case 1 & 2: Node with only one child or no child
        if (root->left == nullptr) {
            Node* temp = root->right;
            delete root;
            return temp;
        } else if (root->right == nullptr) {
            Node* temp = root->left;
            delete root;
            return temp;
        }

        // Case 3: Node with two children
        // Get the inorder successor (smallest in the right subtree)
        Node* temp = findMin(root->right);

        // Copy the inorder successor's content to this node
        root->data = temp->data;

        // Delete the inorder successor
        root->right = deleteNode(root->right, temp->data);
    }
    return root;
}

// Helper function to perform Inorder Traversal (Left, Root, Right)
void inorder(Node* root) {
    if (root != nullptr) {
        inorder(root->left);

    --
```

```
            cout << root->data << " ";
            inorder(root->right);
        }
    }
};

// Driver Code / Execution
int main() {
    BinarySearchTree bst;

    cout << "--- Creation & Insertion ---" << endl;
    bst.root = bst.insert(bst.root, 50);
    bst.insert(bst.root, 30);
    bst.insert(bst.root, 20);
    bst.insert(bst.root, 40);
    bst.insert(bst.root, 70);
    bst.insert(bst.root, 60);
    bst.insert(bst.root, 80);

    cout << "Inorder traversal of the BST:" << endl;
    bst.inorder(bst.root);
    cout << "\n";

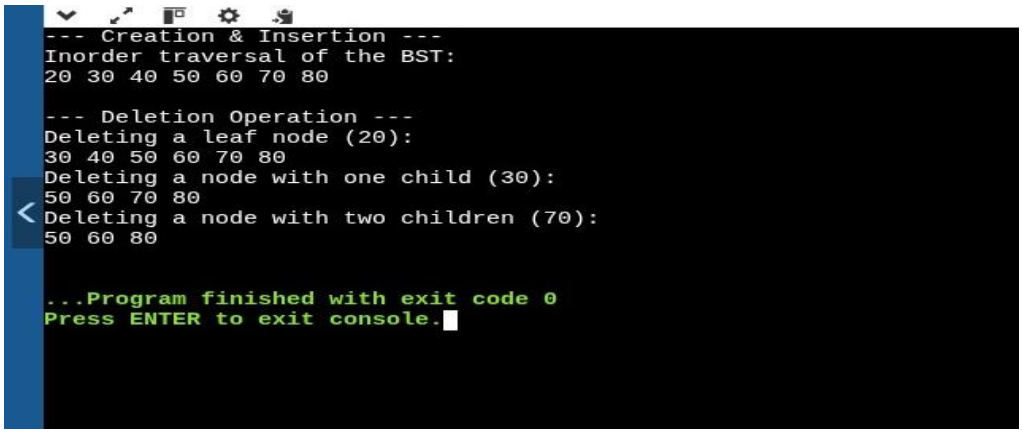
    cout << "\n--- Deletion Operation ---" << endl;

    cout << "Deleting a leaf node (20):" << endl;
    bst.root = bst.deleteNode(bst.root, 20);
    bst.inorder(bst.root);
    cout << "\n";

    cout << "Deleting a node with one child (30):" << endl;
    // Let's first make 30 have only one child by removing 40
    bst.root = bst.deleteNode(bst.root, 40);
    bst.root = bst.deleteNode(bst.root, 30);
    bst.inorder(bst.root);
    cout << "\n";

    cout << "Deleting a node with two children (70):" << endl;
    bst.root = bst.deleteNode(bst.root, 70);
    bst.inorder(bst.root);
    cout << "\n";

    return 0;
}
```


--- Creation & Insertion ---
Inorder traversal of the BST:
20 30 40 50 60 70 80
--- Deletion Operation ---
Deleting a leaf node (20):
30 40 50 60 70 80
Deleting a node with one child (30):
50 60 70 80
Deleting a node with two children (70):
50 60 80
...Program finished with exit code 0
Press ENTER to exit console.

Practical 12

Aim: To implement a Binary Search Tree (BST) and perform Depth-First Search (DFS) traversals including Preorder, Inorder, and Postorder traversals in C++.

Theory:

A **Binary Search Tree (BST)** is a node-based binary tree data structure that has the following properties:

1. The left subtree of a node contains only nodes with keys lesser than the node's key.
2. The right subtree of a node contains only nodes with keys greater than the node's key.
3. The left and right subtree each must also be a binary search tree.

Tree traversal is a process of visiting all the nodes in a tree data structure exactly once in a systematic order. Depth-First Search (DFS) traversals visit nodes deeply along each branch before backtracking.

- **Tree Traversal Techniques:**
  1. **Inorder Traversal (Left, Root, Right):**
     - Process: Visit the left subtree, process the root node, and then visit the right subtree recursively.
     - Feature: For a Binary Search Tree, an Inorder traversal always retrieves elements in **sorted (ascending) order**.
  2. **Preorder Traversal (Root, Left, Right):**
     - Process: Process the root node first, then visit the left subtree, and finally visit the right subtree recursively.
     - Feature: Useful for creating a clone/copy of the tree or creating prefix notation of an expression tree.
  3. **Postorder Traversal (Left, Right, Root):**
     - Process: Visit the left subtree, visit the right subtree, and finally process the root node recursively.
     - Feature: Useful for deleting nodes (bottom-up deletion) or evaluating postfix expressions.

Code Implementation

```
#include <iostream>
using namespace std;

// Structure for a BST Node
struct Node {
    int data;
    Node* left;
    Node* right;

    Node(int val) {
        data = val;
        left = nullptr;
        right = nullptr;
    }
};
```

```
// Class for Binary Search Tree
class BST {
private:
    Node* root;

    // Helper function for recursive insertion
    Node* insertRecursive(Node* node, int val) {
        if (node == nullptr) {
            return new Node(val);
        }
        if (val < node->data) {
            node->left = insertRecursive(node->left, val);
        } else if (val > node->data) {
            node->right = insertRecursive(node->right, val);
        }
        return node;
    }

    // Helper function for Inorder Traversal (Left, Root, Right)
    void inorderRecursive(Node* node) {
        if (node != nullptr) {
            inorderRecursive(node->left);
            cout << node->data << " ";
            inorderRecursive(node->right);
        }
    }

    // Helper function for Preorder Traversal (Root, Left, Right)
    void preorderRecursive(Node* node) {
        if (node != nullptr) {
            cout << node->data << " ";
            preorderRecursive(node->left);
            preorderRecursive(node->right);
        }
    }

    // Helper function for Postorder Traversal (Left, Right, Root)
    void postorderRecursive(Node* node) {
        if (node != nullptr) {
            postorderRecursive(node->left);
            postorderRecursive(node->right);
            cout << node->data << " ";
        }
    }

public:
    BST() {
        root = nullptr;

        --
```

```
    }

    // Public method to insert a key
    void insert(int val) {
        root = insertRecursive(root, val);
    }

    // Public methods to trigger traversals
    void inorder() {
        inorderRecursive(root);
        cout << endl;
    }

    void preorder() {
        preorderRecursive(root);
        cout << endl;
    }

    void postorder() {
        postorderRecursive(root);
        cout << endl;
    }
};

int main() {
    BST tree;

    /* Constructing the following BST:
              50
            /    \
           30     70
          / \   / \
         20  40 60  80
    */

    tree.insert(50);
    tree.insert(30);
    tree.insert(20);
    tree.insert(40);
    tree.insert(70);
    tree.insert(60);
    tree.insert(80);

    cout << "--- BST Traversals ---" << endl;

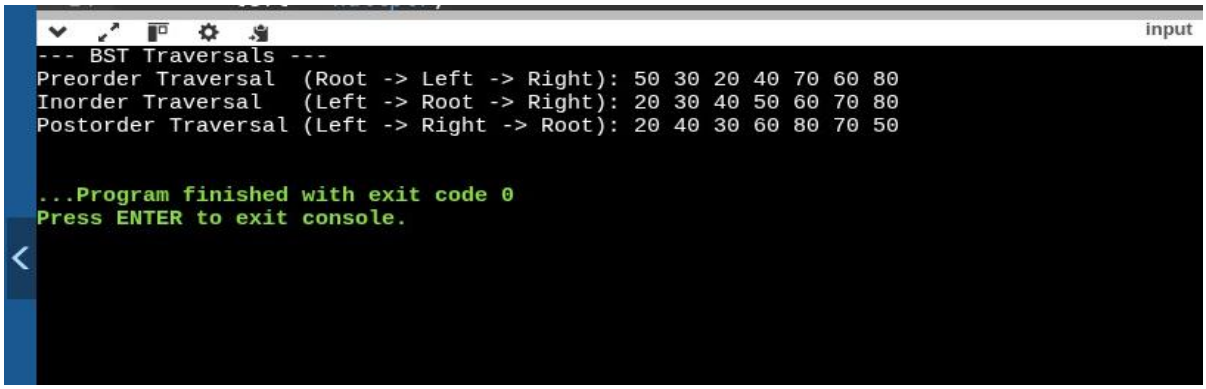
    cout << "Preorder Traversal  (Root -> Left -> Right): ";
    tree.preorder();

    cout << "Inorder Traversal   (Left -> Root -> Right): ";
```

```
    tree.inorder();

    cout << "Postorder Traversal (Left -> Right -> Root): ";
    tree.postorder();

    return 0;
}
```



Practical 13

Aim: To implement Graph data structures, represent them using both Adjacency Matrix and Adjacency List representations, and perform fundamental operations including Graph Creation, Breadth-First Search (BFS) traversal, and Depth-First Search (DFS) traversal in C++.

Theory:

A **Graph** is a non-linear data structure consisting of a set of vertices (or nodes) V and a set of edges E that connect pairs of vertices, represented as G = (V, E).

1. Graph Representations:

- **Adjacency Matrix:** A 2D array of size V x V where entry adj[i][j] is 1 if there is an edge between vertex i and vertex j, and 0 otherwise.
  - *Space Complexity:* $O(V^2)$
  - *Pros:* Constant time O(1) lookup to check if an edge exists between two nodes.
  - *Cons:* Requires $O(V^2)$ memory space regardless of the number of edges.
- **Adjacency List:** An array of lists (or vectors) where each index i contains a list of all vertices adjacent to vertex i.
  - *Space Complexity:* O(V + E)
  - *Pros:* Memory-efficient for sparse graphs; easy to iterate through all neighbors of a node.
  - *Cons:* Edge existence lookup takes O(degree of vertex).

2. Graph Traversals:

- **Breadth-First Search (BFS):**
  - A level-order traversal algorithm that explores graph vertices layer by layer starting from a source vertex.
  - It uses a **Queue** (FIFO data structure) and a **Visited array** to prevent processing the same node multiple times.
  - *Time Complexity:* O(V + E)
- **Depth-First Search (DFS):**
  - A traversal algorithm that explores as deep as possible along each branch before backtracking.
  - It can be implemented using **Recursion** (system call stack) or an explicit **Stack** along with a **Visited array**.
  - *Time Complexity:* O(V + E)

Code Implementation

```
#include <iostream>
#include <vector>
#include <queue>

using namespace std;

class Graph {
private:
```

```
    int numVertices;
    vector<vector<int>> adjMatrix; // Adjacency Matrix representation
    vector<vector<int>> adjList;   // Adjacency List representation

    // Helper recursive function for DFS
    void DFSUtil(int vertex, vector<bool>& visited) {
        visited[vertex] = true;
        cout << vertex << " ";

        // Traverse all adjacent vertices using Adjacency List
        for (int neighbor : adjList[vertex]) {
            if (!visited[neighbor]) {
                DFSUtil(neighbor, visited);
            }
        }
    }

public:
    // Constructor
    Graph(int vertices) {
        numVertices = vertices;
        adjMatrix.resize(vertices, vector<int>(vertices, 0));
        adjList.resize(vertices);
    }

    // Function to add an undirected edge between src and dest
    void addEdge(int src, int dest) {
        // Update Adjacency Matrix
        adjMatrix[src][dest] = 1;
        adjMatrix[dest][src] = 1;

        // Update Adjacency List
        adjList[src].push_back(dest);
        adjList[dest].push_back(src);
    }

    // Display Adjacency Matrix
    void displayAdjacencyMatrix() {
        cout << "Adjacency Matrix:" << endl;
        for (int i = 0; i < numVertices; i++) {
            for (int j = 0; j < numVertices; j++) {
                cout << adjMatrix[i][j] << " ";
            }
            cout << endl;
        }
    }

    // Display Adjacency List
    void displayAdjacencyList() {

        --
```

```
        cout << "Adjacency List:" << endl;
        for (int i = 0; i < numVertices; i++) {
            cout << i << " -> ";
            for (int neighbor : adjList[i]) {
                cout << neighbor << " ";
            }
            cout << endl;
        }
    }

    // Breadth-First Search Traversal
    void BFS(int startVertex) {
        vector<bool> visited(numVertices, false);
        queue<int> q;

        visited[startVertex] = true;
        q.push(startVertex);

        cout << "BFS Traversal starting from vertex " << startVertex << ": ";

        while (!q.empty()) {
            int current = q.front();
            cout << current << " ";
            q.pop();

            for (int neighbor : adjList[current]) {
                if (!visited[neighbor]) {
                    visited[neighbor] = true;
                    q.push(neighbor);
                }
            }
        }
        cout << endl;
    }

    // Depth-First Search Traversal
    void DFS(int startVertex) {
        vector<bool> visited(numVertices, false);
        cout << "DFS Traversal starting from vertex " << startVertex << ": ";
        DFSUtil(startVertex, visited);
        cout << endl;
    }
};

int main() {
    int vertices = 5;
    Graph g(vertices);

    // Creating an example graph

        --
```

```
    // 0 --- 1
    // |  \  |
    // |   \ |
    // 2 --- 3 --- 4
    g.addEdge(0, 1);
    g.addEdge(0, 2);
    g.addEdge(0, 3);
    g.addEdge(1, 3);
    g.addEdge(2, 3);
    g.addEdge(3, 4);

    cout << "--- Graph Representations ---" << endl;
    g.displayAdjacencyMatrix();
    cout << endl;
    g.displayAdjacencyList();

    cout << "\n--- Graph Traversals ---" << endl;
    g.BFS(0);
    g.DFS(0);

    return 0;
}
```

